



MANUAL TEST GUIDE · MODULE 01 OF 11

Masters

Functional-Flow Test Walkthroughs

Scenario-driven QA guide for **logistics.masters** — the governed reference-data foundation the whole domain reads from: UN/LOCODE trade locations, transport modes, equipment & facility masters, commodities, and the carrier network. Here you **set up and govern** the masters the Globex INMUM → SGSIN lane needs, rather than run a transaction.

How to use: Work the scenarios top to bottom against a tenant loaded with `--with-demo=all`. Each step gives a precondition, an action, the expected result, and a ✓ **checkpoint** to tick. Green = happy path; amber/red steps deliberately trip a real validator to confirm it holds.

01 What Masters does & how to read this guide

logistics.masters is the domain's reference-data layer — 42 registered masters that every downstream module (pricing, sales-crm, booking, shipment, operations, tracking) references via `fields.Reference("logistics....")`. It is **read-mostly**: most masters have no commands and are driven through generic CRUD. The exceptions are the **carrier network** (approval governance) and two **sync-not-seed** masters (UN/LOCODE, sailing schedule). Cross-ERP masters — country, city, UOM, party, currency — live in the platform (`res.*`), not here.

42

MASTERS REGISTERED

3

COMMANDS TO EXERCISE

3

TEST SCENARIOS

5

REAL VALIDATOR HOOKS

The three real commands (everything else is generic CRUD)

```
carrier governance : logistics.carrier.approve → approved+active · emits logistics.carrier.approved
                  : logistics.carrier.reject   → rejected+inactive · emits logistics.carrier.rejected
schedule sync      : logistics.sailing.schedule.sync → idempotent upsert of sailings (sync-not-seed)
```

Two classifications you set on the carrier network

CARRIER_TYPE (LOGISTICS.CARRIER)

- `shipping_line` · `airline` · `road_transporter`
- `rail_operator` · `nvocc`
- `mode_id` is **required**; identity is delegated to `res.partner`

APPROVAL_STATE (FEATURE 08 GOVERNANCE)

- `draft` → `submitted` → `approved` | `rejected`
- Default **approved** — governance is **off** unless opted in
- Opt in via `masters.governance.enabled_models`

The scenarios in this guide

Scenario	What it proves	Key mechanism	Section
A — Seed & verify lane masters	Ports INMUM/SGSIN, sea mode, trade lane, FAK commodity exist and are referenceable by downstream modules	<code>ede.search</code> · display-name format	§02
B — Carrier network onboarding	Delegated carrier/provider create, the maker-checker governance gate, approve / reject with real status transitions + events	<code>carrier.approve</code> / <code>.reject</code>	§03
C — Validation guards (negative)	Trip the real pre-create/update validator hooks on <code>vendor.service</code> and <code>haz.class</code> ; confirm the exact refusal messages	lifecycle hooks	§04

Demo fixtures you start from (loaded by `--with-demo=all`)

Ports `masters.demo_port_inmum` (INMUM) + `masters.demo_port_sgsin` (SGSIN); sea mode `masters.transport_mode_sea` ; lane `masters.demo_lane_inmum_sgsin` (transit 7 d); carrier **Maersk Line** `masters.demo_carrier_maersk` (MAEU); NVOCC provider `masters.demo_provider_nvocc` ; commodity **FAK** `masters.demo_commodity_fak` ; DG commodity **LIBAT** (UN3480). UN/LOCODE + sailing schedules are **sync-not-seed** — the two demo ports exist only because the demo tenant has no sync configured.

02 Seed & verify the lane's masters

Goal: confirm the reference data the Globex **INMUM** → **SGSIN** lane depends on is present, well-formed, and **referenceable by downstream modules** (pricing / sales-crm / booking all `ref=` these rows). **Role:** Logistics admin. **Start state:** tenant loaded with `--with-demo=all`.

#	Action	Expected result	✓ Checkpoint	Trace
1	Open Administration → Facilities / Transport & Equipment and read the two lane ports (<code>logistics.unlocode.master</code>).	Rows INMUM (Mumbai, India) and SGSIN (Singapore) exist; each renders as [INMUM] Mumbai / [SGSIN] Singapore — the <code>display_name_format</code> " <code>{code} {name}</code> ".	Both ports present; code shows in the label.	demo_ports
2	Open the sea transport mode (<code>logistics.transport.mode.master</code>).	Row SEA present with the system-trusted <code>mode_type=sea</code> (fixed enum: sea/air/road/rail).	mode_type = sea, not free text.	seed CSV
3	Open the demo trade lane (<code>logistics.trade.lane</code> , id <code>masters.demo_lane_inmum_sgsin</code>).	Lane binds origin INMUM → destination SGSIN over <code>transport_mode_id=sea</code> , <code>standard_transit_days=7</code> .	Lane endpoints + mode resolve to the seeded masters.	demo
4	Open the baseline commodity (<code>logistics.commodity.master</code> , id <code>masters.demo_commodity_fak</code>).	FAK — Freight All Kinds , HS 9999, <code>stackable=true</code> , <code>dg_flag=false</code> . Renders as [FAK] Freight All Kinds .	FAK present; not flagged DG.	demo
5	Confirm an equipment master row is available for load planning (<code>logistics.equipment.type.master</code> catalogue).	A container equipment type is resolvable for the lane's FCL cargo (dimensions / payload datasheet on the row).	Equipment type resolvable for FCL.	seed CSV
6	Referenceability check: open the pricing (or sales-crm) demo and confirm its rate / quote resolves these lane endpoints.	The downstream record's origin/destination/mode/commodity FKs point at exactly the rows verified above — no free-text drift.	Downstream record references the same master rows.	FK

✓ Scenario A pass criteria

Every lane master exists, carries its system-trusted enum / display code, and is bound by FK from at least one downstream demo record — proving the masters are the single source of truth the domain reads from.

⚠ UN/LOCODE & sailing schedule are sync-not-seed

In production the UN/LOCODE catalogue is **synced on demand** from an external reference platform — it is **never CSV/XML-seeded**. The two demo ports exist only because the demo tenant has no sync configured. Likewise sailing schedules are populated only via `logistics.sailing.schedule.sync` — consumers **search** synced rows, never hand-create them. Do not treat the demo ports as the production seeding pattern.

03 Carrier network onboarding & approval governance

Goal: onboard a carrier/provider and prove the **maker-checker governance gate** — new master rows can be held **submitted + inactive** until an approver acts, so unreviewed reference data never reaches transactions. **Role:** Logistics admin (maker) + governance approver (checker). **Start:** governance **off** by default.

approval_state : draft → submitted → approved (active=true) | submitted → rejected (active=false)
governance gate: opt in per master via ir.config **masters.governance.enabled_models** = "logistics.carrier"

#	Action	Expected result	✓ Checkpoint	Trace
1	With governance off , create a carrier (<code>ede.create</code> on <code>logistics.carrier</code>) with only <code>name</code> , <code>carrier_type=shipping_line</code> , <code>mode_id=sea</code> .	Carrier created live. Identity is delegated to res.partner — a partner shadow is auto-created; the pre-create hook defaults the shadow's <code>country_id</code> + <code>email</code> so no partner fields need typing. <code>approval_state=approved</code> , <code>active=true</code> .	Carrier live; partner shadow auto-created.	F01 hook
2	Set ir.config <code>masters.governance.enabled_models = logistics.carrier</code> , then create a second carrier.	The governance gate fires: the new row is forced <code>approval_state=submitted</code> and <code>active=false</code> — it will not surface to transactions until approved.	New carrier parks at submitted, inactive.	F08 gate
3	As the approver, run <code>logistics.carrier.approve</code> against the submitted carrier.	<code>approval_state=approved</code> , <code>active=true</code> ; a <code>logistics.carrier.approved</code> event fires with the carrier id. Handler returns <code>{ok:true, carrier_id}</code> .	Carrier activated; approved event emitted.	F08 event
4	Create a third carrier (governance on), then run <code>logistics.carrier.reject</code> with a <code>reason</code> .	<code>approval_state=rejected</code> , <code>active=false</code> (stays out of transactions); a <code>logistics.carrier.rejected</code> event fires carrying the reason.	Carrier rejected + inactive; reason on event.	F08 event
5	Onboard a shipping provider (<code>logistics.provider</code>) — e.g. <code>provider_type=nvocc</code> — and a carrier agent (<code>logistics.carrier.agent</code> , <code>agent_type=gsa</code>) under the approved carrier.	Both created; each delegates identity to <code>res.partner</code> with the same country/email defaulting hook. Agent's <code>carrier_id</code> is required and resolves to the carrier.	Provider + agent created, partner shadows present.	F01 hook
6	Run <code>logistics.sailing.schedule.sync</code> with a batch of rows for the lane, then re-run the same batch.	First run creates sailings; the re-run updates in place (idempotent upsert on (<code>source</code> , <code>source_ref</code>)) — no duplicates. Returns <code>{ok, created, updated, total}</code> .	Re-sync produces zero duplicates.	F03 sync

✓ Scenario B pass criteria

Governance off = carriers go live immediately; governance on = new carriers park at **submitted/inactive** until `approve` (→ approved/active + event) or `reject` (→ rejected/inactive + event). Delegated identity means partner shadows appear with no re-keying, and schedule sync is idempotent.

⚠ Approve/reject need a target record

Both `approve` and `reject` operate on `cmd.record`. Dispatching either with no target raises `"logistics.carrier.approve requires a target carrier"` (resp. `"...reject requires a target carrier"`) — always route the command at a concrete carrier row.

04 Validation guards — the refusals that must hold

Half of testing reference data is confirming it **refuses** malformed rows. This module raises on exactly two masters — the **dangerous-goods taxonomy** and the **vendor-service catalogue** — via pre-create / pre-update lifecycle hooks. Each row below trips a real hook and states the **exact** `ValueError` string.

#	Action (bad input)	Expected refusal — exact message	✓	Trace
1	Create a <code>logistics.haz.class</code> with <code>level=division</code> but no <code>parent_id</code> .	Rejected: "A dangerous-goods division requires a parent class (<code>parent_id</code>)."	Blocked	BR-HZ-01 haz create hook
2	Create a <code>logistics.haz.class</code> with <code>level=class</code> but a <code>parent_id</code> set.	Rejected: "A dangerous-goods class is top-level and cannot have a <code>parent_id</code> ."	Blocked	BR-HZ-02 haz create hook
3	Update an existing division to clear its <code>parent_id</code> (write <code>level=division</code> , <code>parent_id=null</code>).	Rejected with the same division message — the update hook enforces the invariant too, not only create.	Blocked on update	BR-HZ-01 haz update hook
4	Partial update of a haz class touching neither <code>level</code> nor <code>parent_id</code> (e.g. rename).	Allowed — the guard only fires when the write includes <code>level</code> or <code>parent_id</code> ; unrelated edits pass.	Rename succeeds	hook scope
5	Create a <code>logistics.vendor.service</code> with <code>valid_to</code> earlier than <code>valid_from</code> .	Rejected: "Vendor service <code>valid_to</code> must be on or after <code>valid_from</code> ."	Blocked	BR-VC-02 vs create hook
6	Update a valid vendor-service row to set <code>valid_to</code> before <code>valid_from</code> .	Rejected with the same message — the update hook re-validates the window.	Blocked on update	BR-VC-02 vs update hook
7	Dispatch <code>logistics.carrier.approve</code> / <code>.reject</code> with no target record .	Rejected: " <code>logistics.carrier.approve</code> requires a target <code>carrier</code> " (resp. <code>...reject...</code>).	Blocked	F08 command guard

⚠ Not every master hook is a validator — carrier / agent / provider create hooks only DEFAULT

The `pre.logistics.carrier.create`, `...carrier.agent.create` and `...provider.create` hooks **do not reject bad input** — they **default** the delegated `res.partner` shadow's `country_id` + `email` (and, for carrier, apply the governance gate). Only `haz.class` and `vendor.service` raise `ValueError`. Do not expect a refusal from a carrier create with a missing partner field — it is silently completed.

WHERE THE GUARDS LIVE

- `haz.class` — `pre....create` + `pre....update` (level ↔ parent invariant)
- `vendor.service` — `pre....create` + `pre....update` (validity window)
- `carrier` — `approve` / `reject` commands guard on a target row

READ-ONLY MASTERS HAVE NO GUARDS

- Most of the 42 masters are pure CRUD reference rows
- `is_standard_locked=true` is a UI contract, **not** a command-layer block
- Commodity's "UN number when dg_flag" is a **docstring rule**, not an enforced hook

✓ Full-module pass criteria

All three scenarios green: lane masters exist & are referenceable (A); carrier governance approve/reject with real status + events (B); and every documented validator (`haz.class` , `vendor.service` , carrier command guard) confirmed to refuse with its exact message (C).

Traceability: BR-HZ-01/02 and BR-VC-02 are the model docstring rule IDs; F01/F03/F08 map to `masters` Phase 2 features 01 (party masters) / 03 (sailing schedule) / 08 (approval governance). Source inputs: `src/domains/logistics/docs/masters.md` + `docs/demo-usecase/modules/logistics-masters/usecase.md` — this module ships no BRD.